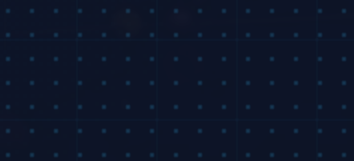


CURSO 2026

PROGRAMACIÓN AVANZADA

Algoritmos, estructuras y técnicas esenciales para el desarrollo de software moderno.

 NOTACIÓN BIG O ESTRUCTURAS DE DATOS ORDENAMIENTO RECURSIVIDAD

Contenido del Curso



SECCIÓN 01

Notación Big O

Mide la eficiencia de algoritmos en tiempo y espacio según el crecimiento de n , describiendo el peor escenario posible.

 Complejidad Algorítmica



SECCIÓN 02

Estructuras de Datos

Pilas, colas, árboles, grafos, tablas hash y matrices: 8 tipos fundamentales para organizar y acceder a la información.

 8 Tipos Fundamentales



SECCIÓN 03

Métodos de Ordenamiento

Comparación de 9 algoritmos: burbuja, inserción, selección, merge sort, quicksort y más, analizando su eficiencia.

 9 Algoritmos Comparados



SECCIÓN 04

Recursividad

Técnica divide y conquista: funciones que se llaman a sí mismas para resolver subproblemas de forma elegante y eficiente.

 Divide y Conquista

Notación **Big O**

Estándar para expresar la eficiencia de un algoritmo en función de n



Complejidad de Tiempo

Mide cuántas operaciones realiza un algoritmo conforme el tamaño de entrada n crece. No mide segundos exactos, sino el crecimiento del trabajo.



Complejidad de Espacio

Cuantifica la memoria adicional que necesita el algoritmo para ejecutarse en función del tamaño de la entrada.



Peor Escenario (Cota Superior)

Big O describe el caso más desfavorable posible — garantiza que el algoritmo nunca superará ese límite de crecimiento.



Ejemplo: Búsqueda en 1,000 Nombres

Búsqueda **binaria** (lista ordenada): ~10 pasos | Búsqueda **lineal** (desordenada): hasta 1,000 pasos.



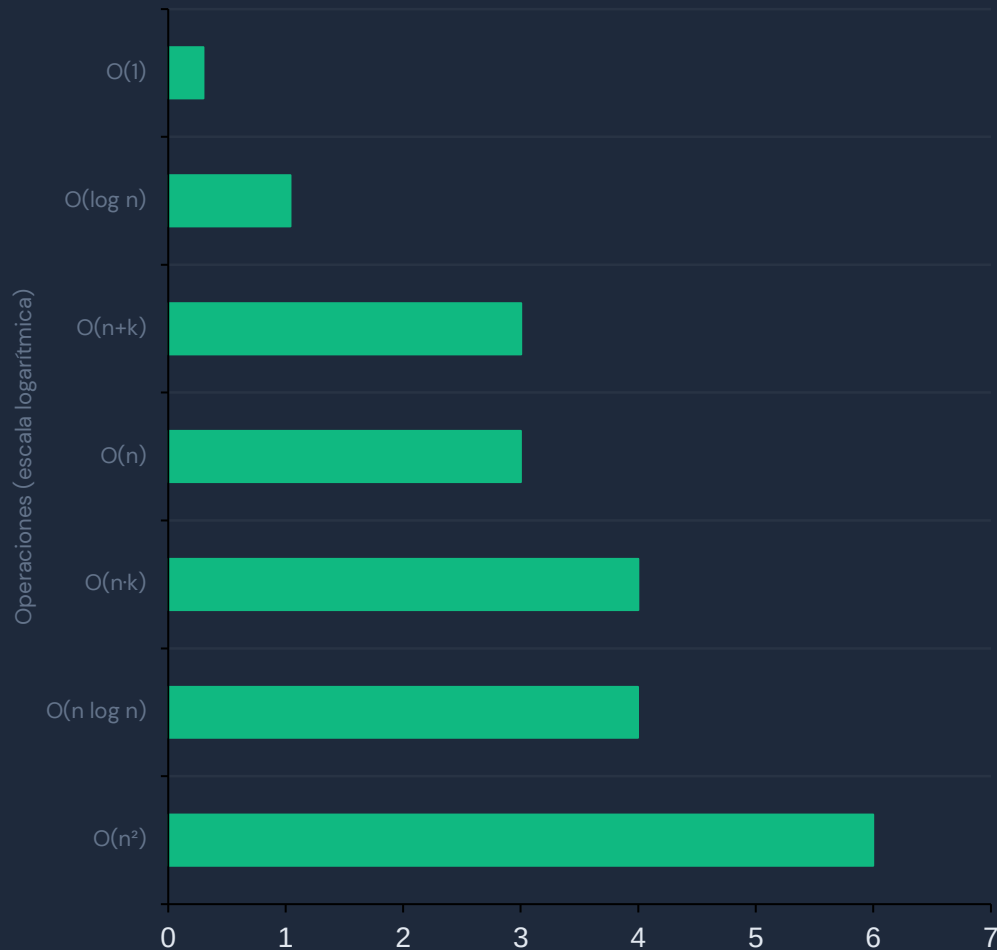
Clave: Big O no mide el tiempo real en segundos — mide **cómo crece el trabajo** al aumentar n , permitiendo comparar algoritmos de forma independiente al hardware.



Notaciones y su Significado

Operaciones para $n = 1,000$ — escala logarítmica

📊 NÚMERO DE OPERACIONES POR NOTACIÓN BIG O



Espectro de Eficiencia

⚡ **$O(1)$ y $O(\log n)$** — Constante y logarítmico son los más eficientes. Ejemplos: acceso a array `arr[5]`, búsqueda binaria y árboles BST.

= **$O(n)$ y $O(n \log n)$** — Crecimiento lineal y lineal-logarítmico. Recorrer listas enlazadas, Merge Sort y Quick Sort entran en esta categoría.

⚠️ **$O(n^2)$** — Cuadrático con dos ciclos anidados. Bubble Sort y Selection Sort generan 1,000,000 operaciones para $n=1,000$.

★ **$O(n+k)$ y $O(n \cdot k)$** — Casos especiales: Counting Sort y Radix Sort superan los comparativos cuando el rango k es pequeño.

💡 **Conclusión clave:** La diferencia entre $O(n \log n)$ y $O(n^2)$ es de $\sim 100\times$ para $n=1,000$ — elegir el algoritmo correcto es crítico a escala.

¿Cómo Elegir un Algoritmo?

No existe un algoritmo universalmente óptimo; la elección correcta depende del contexto, las restricciones del problema y las características de los datos.

01 / CRITERIO

Tamaño de Datos

Para listas con **menos de 50 elementos**, Insertion Sort puede superar a Merge Sort gracias a su bajo overhead de implementación y ausencia de recursión.

02 / CRITERIO

Datos Casi Ordenados

Insertion Sort alcanza **$O(n)$** cuando los datos ya están parcialmente ordenados, convirtiéndolo en la opción más eficiente en ese escenario específico.

03 / CRITERIO

Tiempo vs. Espacio

Merge Sort ofrece **$O(n \log n)$** pero requiere $O(n)$ memoria auxiliar. Heap Sort logra la misma complejidad temporal con **$O(1)$** espacio adicional.

04 / CRITERIO

Estabilidad

Si el orden relativo de elementos iguales debe preservarse, se debe elegir un algoritmo **estable** como Merge Sort o Insertion Sort, no Heap Sort ni Quick Sort.

05 / CRITERIO

Rango Limitado

Counting Sort con complejidad **$O(n+k)$** supera a cualquier algoritmo comparativo cuando el rango de valores k es pequeño y conocido de antemano.

CONCLUSIÓN CLAVE Conocer el contexto del problema es tan importante como conocer la complejidad teórica del algoritmo.

Estructuras de Datos Lineales

STACK VS QUEUE — COMPARATIVA DE ESTRUCTURAS LINEALES



Pila (Stack)

LIFO — LAST IN, FIRST OUT



push () $O(1)$



pop () $O(1)$



peek () $O(1)$



Recursión: La pila de llamadas del sistema gestiona funciones recursivas en orden LIFO.



Ctrl+Z: Deshacer acciones usa una pila para revertir cambios en orden inverso.



Historial: El navegador apila páginas visitadas; el botón Atrás hace pop.



Balanceo: Verificación de paréntesis y llaves en compiladores e intérpretes.



Pila de platos — el último en entrar es el primero en salir



Espacio total: $O(n)$ — proporcional al número de elementos



Cola (Queue)

FIFO — FIRST IN, FIRST OUT



enqueue () $O(1)$



dequeue () $O(1)$



front () $O(1)$



Planificador SO: El sistema operativo gestiona procesos en cola para asignar tiempo de CPU.



Impresión: Los trabajos se imprimen en el orden en que fueron enviados.



Peticiones web: Los servidores atienden solicitudes HTTP en orden de llegada.



BFS en grafos: La búsqueda en anchura usa una cola para explorar nodos nivel por nivel.



Fila del supermercado — el primero en llegar es el primero en ser atendido



Espacio total: $O(n)$ — proporcional al número de elementos

VS

Listas Enlazadas

Nodos conectados mediante referencias — tres variantes fundamentales



Lista Simple UNIDIRECCIONAL

Cada nodo contiene **valor + referencia al siguiente**.
Complejidad: **insertAtBeginning $O(1)$** · insertAtEnd $O(n)$ · search $O(n)$

● Pilas y colas · Historial de navegación · Listas de reproducción



Lista Doblemente Enlazada BIDIRECCIONAL

Nodo con **valor + puntero anterior + puntero siguiente**. Permite recorrer en ambas direcciones con mayor flexibilidad.

● Navegadores web (adelante/atrás) · Listas musicales · Editores de texto



Lista Circular CICLO CONTINUO

El **último nodo apunta al primero**, formando un ciclo cerrado. No existe un nodo final nulo — la lista se recorre indefinidamente.

● Juegos por turnos · Round-Robin scheduling · Buffers circulares



Clave: Las listas enlazadas permiten inserciones/eliminaciones eficientes en $O(1)$ al inicio, pero el acceso aleatorio es $O(n)$ — a diferencia de los arrays con acceso $O(1)$.

Árboles & Grafos

ESTRUCTURAS AVANZADAS



Árboles — Estructura

Nodos conectados por aristas en jerarquía. Raíz, hijos y hojas. Cada nodo tiene un único padre.

Jerárquico



Recorridos del Árbol

Inorden Izq→Raíz→Der · **Preorden**
Raíz→Izq→Der · **Postorden** Izq→Der→Raíz

DFS Traversal



Usos de Árboles

BST para búsqueda ordenada, expresiones matemáticas, índices en bases de datos, compiladores.

BST · Expresiones · BD



Grafos — Estructura

Vértices conectados por aristas. Pueden ser **dirigidos** o **no dirigidos**, con o sin pesos.

Dirigido · No Dirigido



Algoritmos de Camino

Dijkstra pesos positivos · **Bellman-Ford** pesos negativos · **Floyd-Warshall** todos los pares · **A*** heurísticas



Usos de Grafos

Rutas GPS, redes sociales, circuitos eléctricos, videojuegos y sistemas de recomendación.

GPS · Redes · Circuitos



Redes de Nodos

Representación visual de grafos y árboles

COMPLEJIDADES CLAVE

$O(\log n)$

Búsqueda en BST balanceado

$O(V^2)$

Dijkstra con matriz de adyacencia

$O(V + E)$

Recorrido BFS / DFS en grafos

$O(V^3)$

Floyd-Warshall todos los pares

Tablas Hash y Matrices

Tabla Hash

$O(1)$ promedio

Usa una **función hash** para mapear claves a valores en memoria.
Operaciones principales: **put(clave, valor)**, **get(clave)** y **remove(clave)**.
Caso promedio $O(1)$; peor caso $O(n)$ por colisiones.

put / get / remove

Peor caso $O(n)$

Mapeo clave→valor

🔗 Manejo de Colisiones

4 métodos

Cuando dos claves generan el mismo índice, se aplica una estrategia de resolución. **Encadenamiento**: lista enlazada por celda. **Sondeo lineal** y **cuadrático**: buscan la siguiente celda libre. **Doble hashing**: segunda función hash para el desplazamiento.

Encadenamiento

Sondeo lineal

Sondeo cuadrático

Doble hashing

📊 Matrices 2D (Array $n \times m$)

$O(1)$ acceso

Estructura bidimensional de **n filas × m columnas**. Acceso directo con **matriz[i][j]** en tiempo $O(1)$. Recorrido completo de todos los elementos en $O(n \cdot m)$. Ideal para representar datos tabulares y espaciales.

matriz[i][j] → $O(1)$

Recorrido $O(n \cdot m)$

Estructura bidimensional

🗄 Usos Reales

Aplicaciones

Hash Tables: índices en bases de datos, cachés como **Redis**, tablas de símbolos en compiladores. **Matrices**: tableros de juegos (ajedrez), imágenes digitales (píxeles), multiplicación matricial y **matriz de adyacencia** para grafos.

Redis / Caché

Compiladores

Imágenes digitales

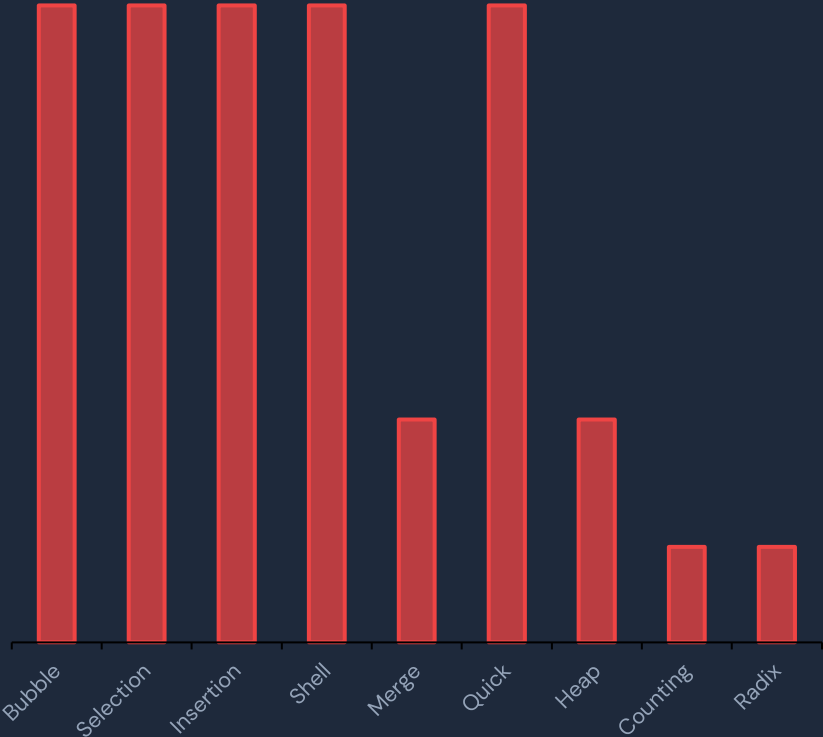
Grafos

Comparativa de Algoritmos de Ordenamiento

O(N²) — SIMPLES O(N LOG N) — EFICIENTES NO COMPARATIVOS

Algoritmo	Mejor Caso	Caso Promedio	Peor Caso	Espacio	Estable
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	✓
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	✗
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	✓
Shell Sort	O(n log n)	O(n log²n)	O(n²)	O(1)	✗
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	✓
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	✗
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	✗
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	✓
Radix Sort	O(n•k)	O(n•k)	O(n•k)	O(n+k)	✓

COSTO RELATIVO — PEOR CASO



⚠ O(n²) — Simples pero lentos; solo para listas pequeñas o educación.

⚡ O(n log n) — Eficientes para uso general; Quick Sort es el más usado.

∞ No comparativos — Superan O(n log n) con enteros de rango limitado.

Algoritmos de Ordenamiento **Simple**s

Fundamentos esenciales — complejidad $O(n^2)$ con casos de uso específicos



Bubble Sort

$O(n^2)$

✓ Estable

Compara **elementos adyacentes** e intercambia si están en orden incorrecto. El mayor "burbujea" al final en cada pasada. **Optimización:** si no hay intercambios, el arreglo ya está ordenado.

💡 **Cuándo usar:** Propósitos educativos y listas muy pequeñas



Selection Sort

$O(n^2)$

✗ No estable

Divide el arreglo en parte **ordenada / no ordenada**, encuentra el mínimo y lo intercambia. Siempre $O(n^2)$ comparaciones pero solo **$O(n)$ intercambios** — ventaja clave.

💡 **Cuándo usar:** Cuando escribir en memoria es costoso



Insertion Sort

$O(n) / O(n^2)$

✓ Estable

Construye el arreglo ordenado **uno a uno** (como ordenar cartas en la mano). Mejor caso **$O(n)$** con datos casi ordenados — el más eficiente de los simples en ese escenario.

💡 **Cuándo usar:** Datos en tiempo real o listas de menos de 50 elementos



Shell Sort

$O(n \log n) / O(n^2)$

✗ No estable

Mejora de Insertion Sort usando un **gap (intervalo)** decreciente. Ordena elementos distantes primero, reduciendo el gap progresivamente **hasta 1** para finalizar con Insertion Sort.

💡 **Cuándo usar:** Mejora práctica de Insertion Sort para listas medianas

Algoritmos Eficientes

⚡ $O(n \log n)$ y superiores



Merge Sort

$O(n \log n)$

Divide y conquista: divide en mitades, ordena recursivamente y combina. Complejidad garantizada en todos los casos.

✓ Estable

$O(n)$ espacio

Listas enlazadas



Quick Sort

$O(n \log n)^*$

Elige un pivote y particiona: menores a izquierda, mayores a derecha. Muy rápido en la práctica con $O(\log n)$ espacio.

✗ No estable

Más usado

Peor $O(n^2)$



Heap Sort

$O(n \log n)$

Basado en árbol binario max-heap. Complejidad garantizada $O(n \log n)$ con $O(1)$ espacio extra (in-place).

✗ No estable

$O(1)$ espacio

Memoria limitada



Counting / Radix Sort

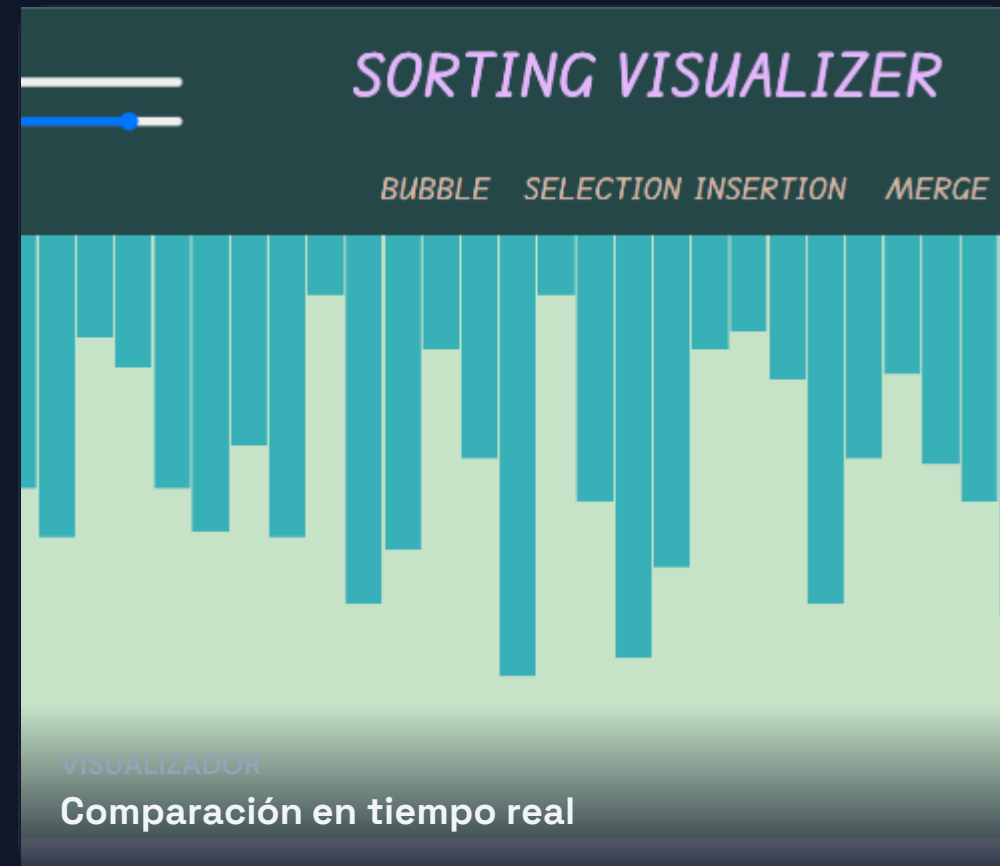
$O(n+k)$

Algoritmos no comparativos que superan la barrera $O(n \log n)$. Counting Sort $O(n+k)$, Radix Sort $O(n \cdot k)$.

✓ Estable

No comparativo

Enteros



VENTAJAS CLAVE

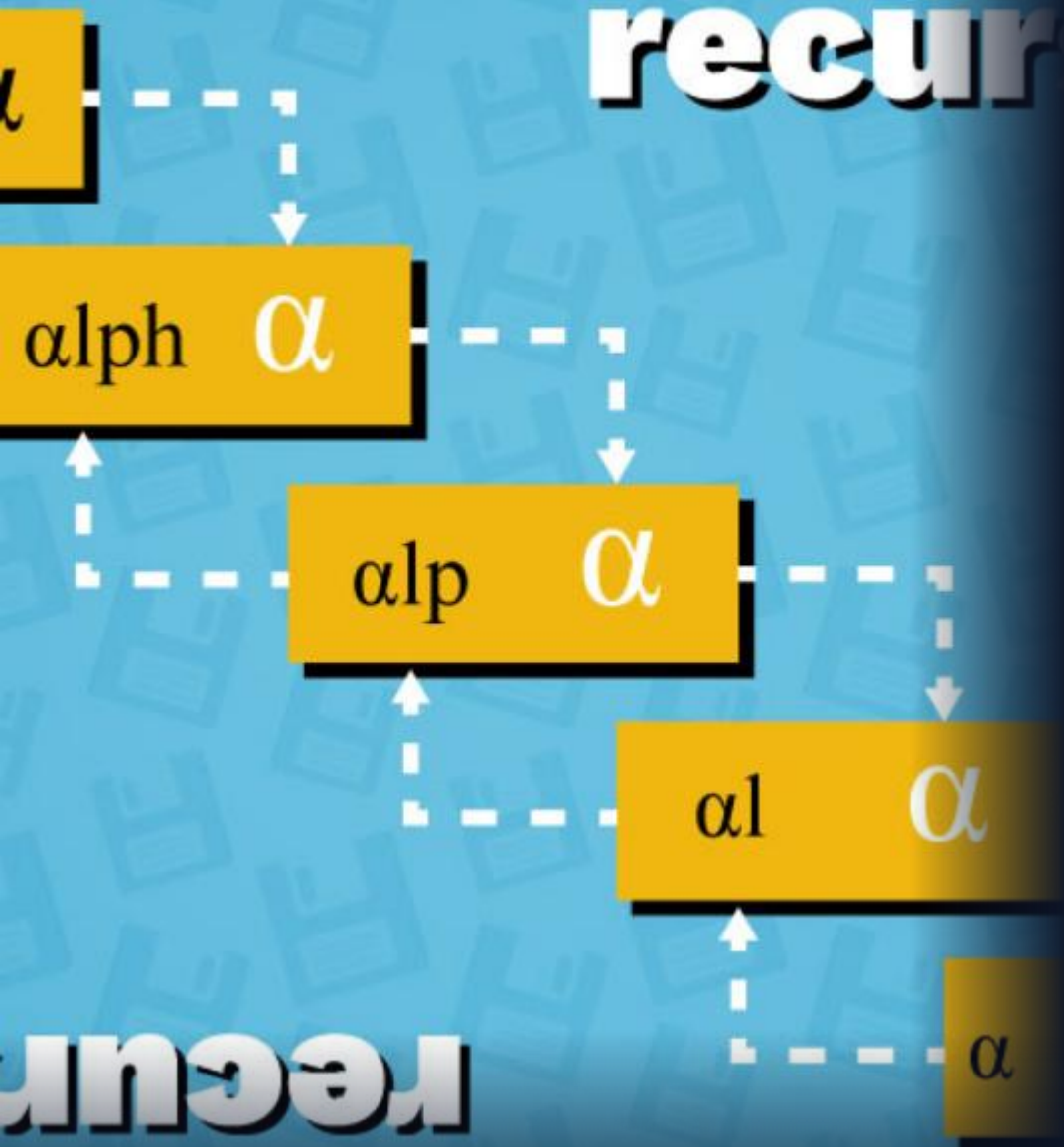
Garantizado

$O(n \log n)$ en Merge y Heap Sort

Superan

Counting/Radix rompen la barrera $O(n \log n)$

recur



recur

> PROGRAMACIÓN · FUNCIONES

Recursividad

Una función que se llama a sí misma para resolver subproblemas

🟢 CASO BASE

Condición que **detiene** la recursión y evita llamadas infinitas. Debe definirse claramente.

🔄 CASO RECURSIVO

La función se llama a sí misma **acercándose** al caso base en cada iteración.

📦 PILA DE LLAMADAS LIFO

Cada llamada se **apila**. Al alcanzar el caso base, se resuelven en orden inverso (desapilando).

</> EJEMPLOS CLÁSICOS

Factorial **Fibonacci** **Torres de Hanoi**
Merge Sort

⚡ **Memoización:** Almacena resultados de llamadas previas para evitar cálculos repetidos — transforma Fibonacci de complejidad $O(2^n)$ a $O(n)$.

Recursividad vs Iteración

Comparación en 6 aspectos clave — elige la herramienta correcta para cada problema

Recursividad

-  **Legibilidad**
Código más limpio y declarativo, expresa la solución de forma natural
-  **Memoria**
Consume más memoria por la pila de llamadas (call stack)
-  **Rendimiento**
Puede ser más lento debido al overhead de cada llamada
-  **Stack Overflow**
Riesgo real con muchas llamadas anidadas sin caso base correcto
-  **Problemas Ideales**
Árboles, grafos, divide y conquista, Torres de Hanoi
-  **Facilidad**
Natural para problemas con estructura recursiva inherente

vs

Iteración

-  **Legibilidad**
Puede ser más verboso, pero familiar y directo para la mayoría
-  **Memoria**
Más eficiente, usa memoria constante $O(1)$ en la mayoría de casos
-  **Rendimiento**
Generalmente más rápido, sin overhead de llamadas a funciones
-  **Stack Overflow**
Sin riesgo — los bucles no acumulan frames en la pila
-  **Problemas Ideales**
Cálculos repetitivos, recorridos lineales, procesamiento de listas
-  **Facilidad**
Requiere pensar en bucles, contadores y condiciones de parada



Conclusión: Elige **recursividad** para elegancia y claridad en problemas jerárquicos; usa **iteración** para eficiencia y seguridad en cálculos repetitivos. La memoización puede combinar lo mejor de ambos mundos.